

控制 时钟频率 = 10_000_000

PUB go() | i

调试 (位图 MyBitmap 尺寸 32 16 点尺寸 8 LUT2 LONGS_2BIT)

调试 (MyBitmap 追踪 14 LUT 颜色 白色 红色 蓝色 黄色 6)

重复

调试 (MyBitmap uhex_(标志 [i++ & \$1F]) dly(100))

数据

标志

长

已处理。

长

``zh%%00101010102222220``

长

%%001010101020202020

长

``zh%%00101010102222220``

长

%%001010101022020220

长

``zh%%00101010102222220``

长

%%001010101020202020

长

``zh%%00101010102222220``

长

%%001010101022020220

长

``zh%%00101010102222220``

长

%%001010101020202020

长

``zh%%00101010102222220``

长

%%001010101010101010

长

%%001010101010101010

长

%%001010101010101010

长

%%001010101010101010

长

%%001010101010101010

长

%%001010101010101010

长

%%001010101010101010

长

%%001010101010101010

长

%%001010101010101010

长

%%001010101010101010

长

%%001010101010101010

长

%%001010101010101010

长

已处理。

长

%%000000000000000000

长

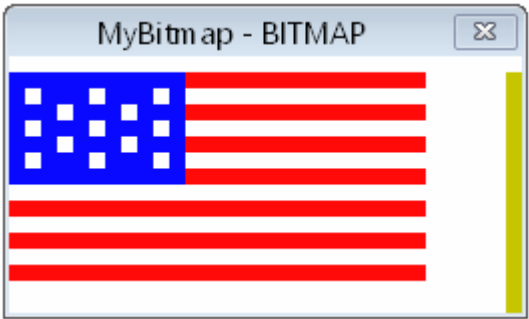
000000000000

长

000000000000

长

%%000000000000000000



位图实例化	描述	默认
标题 ' 字符串 '	将窗口标题设置为 “ 字符串 ”。	无
位置 左上	设置窗口位置。	0, 0
尺寸 x_ 像素 y_ 像素	设置位图中像素的数量（ x 和 y 均为 1..2048 ）。	256, 256
DOTSIZE x_ 和 _y_ 尺寸 {y_ 尺寸}	设置显示像素的尺寸 (1..256)。‘DOTSIZE 16’ 使最终显示中的每个像素为 16x16。	1,

TRACE modes

Rate is set to 1 so that each pixel can be seen as it's loaded.

0

1

2

3

4

5

6

7

8

9

10

11

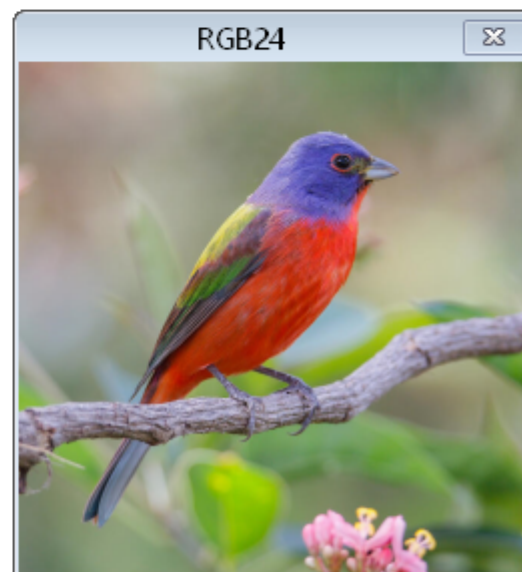
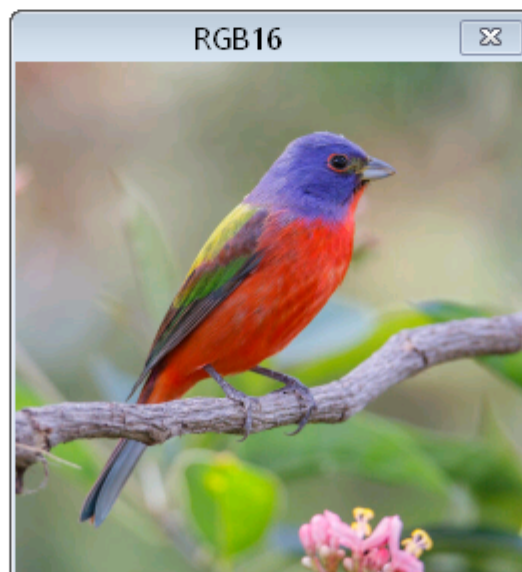
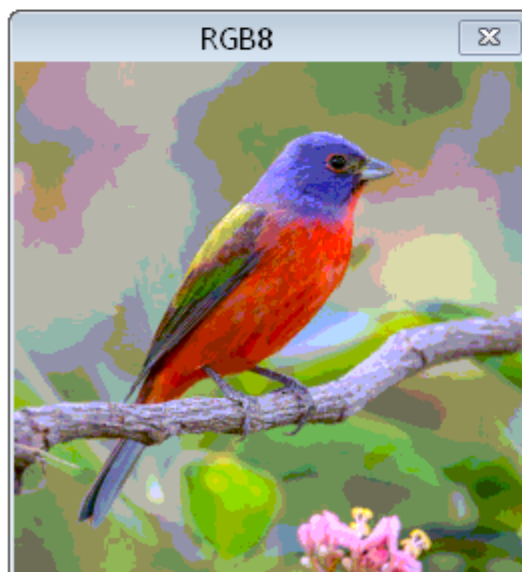
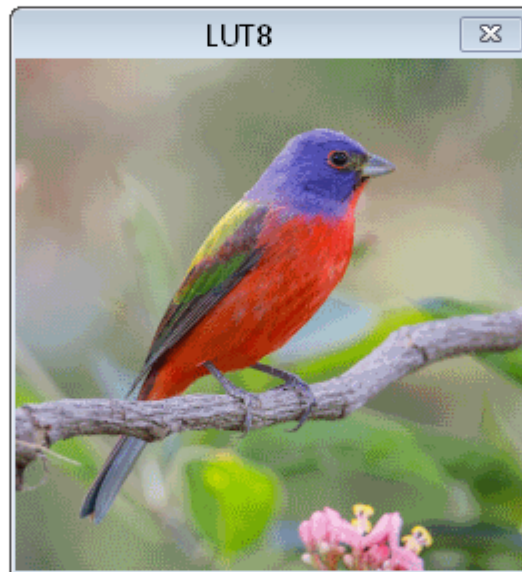
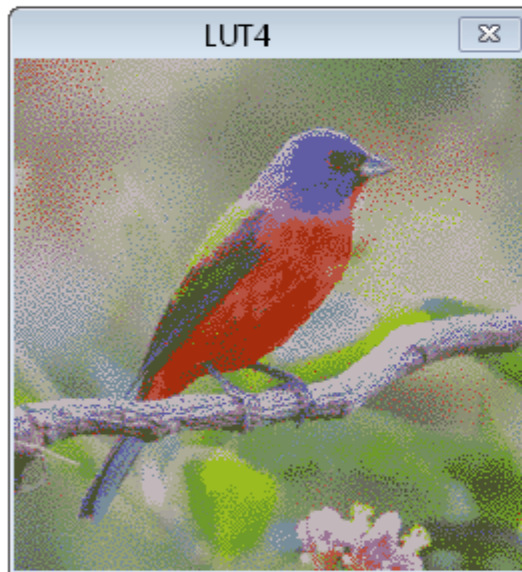
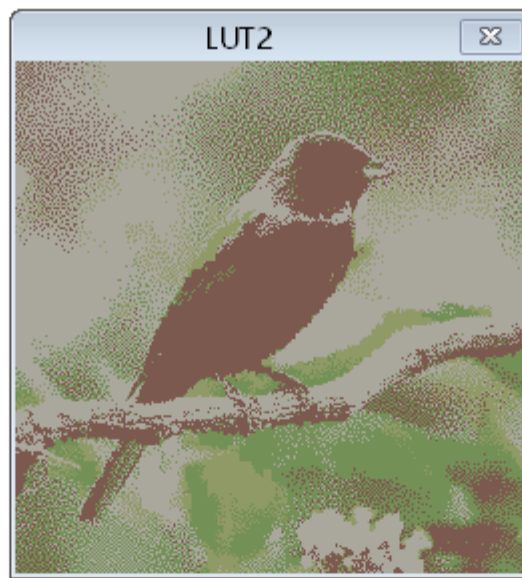
12

13

14

15

颜色模式	位像素	描述	意图
L UT1	1	像素索引 LUT 颜色 0..1	内存效率高 2 色调色板图形
L UT2	2	像素索引 LUT 颜色 0..3	Memory-efficient 4-color-palette graphics
L UT4	4	像素索引 LUT 颜色 0..15	高效的 16 颜色 调色板 图形
L UT8	8	像素索引 LUT 颜色 0..255	高效的 256 颜色 调色板 图形。
LUMA8	8	从黑色到颜色 *	仪器，其中亮度表示水平
LUMA8W	8	从白色到颜色 *	仪器，其中饱和度表示电平
LUMA8X	8	从黑色到颜色 * 到白色	仪器，亮度指示水平，峰值在白色
HSV8	8	从黑色到颜色：%HHHHSSSS	16 色调，16 亮度级别
HSV8W	8	从白色到颜色：%HHHHSSSS	16 种色调，包含 16 种饱和度级别，源自白色
HSV8X	8	从黑色到颜色到白色：%HHHHSSSS	16 种色调，包含 16 种亮度级别，峰值在白色。
RGBI8	8	从黑色到颜色：%RGBIIIII	8 种基本颜色，包含 32 个亮度级别
RGBI8 W	8	白色到颜色：%RGBIIIII	8 种基本颜色，有 32 个饱和度级别，从白色而来。
RGBI8X	8	从黑色到颜色到白色：%RGBIIIII	8 种基本颜色，具有 32 级亮度，峰值在白色。
RGB8	8	%RRRGGBBB	字节级 RGB，包含 8 级红色、8 级绿色和 4 级蓝色
HSV16	16	从黑色到颜色：%HHHHHHHHH_SSSSSSSS	256 种色调和 256 级亮度
HSV16W	16	从白色到颜色：%HHHHHHHHH_SSSSSSSS	256 种色调，包含 256 种饱和度级别，源自白色
HSV16X	16	从黑色到颜色到白色：%HHHHHHHHH_SSSSSSSS	256 种色调，256 级亮度，峰值在白色
RGB16	16	%RRRRRGGG_GGGBBBBB	32 红色级别、64 绿色级别和 32 蓝色级别的字元级 RGB
RGB24	24	%	



```

CON _clkfreq = 100_000_000

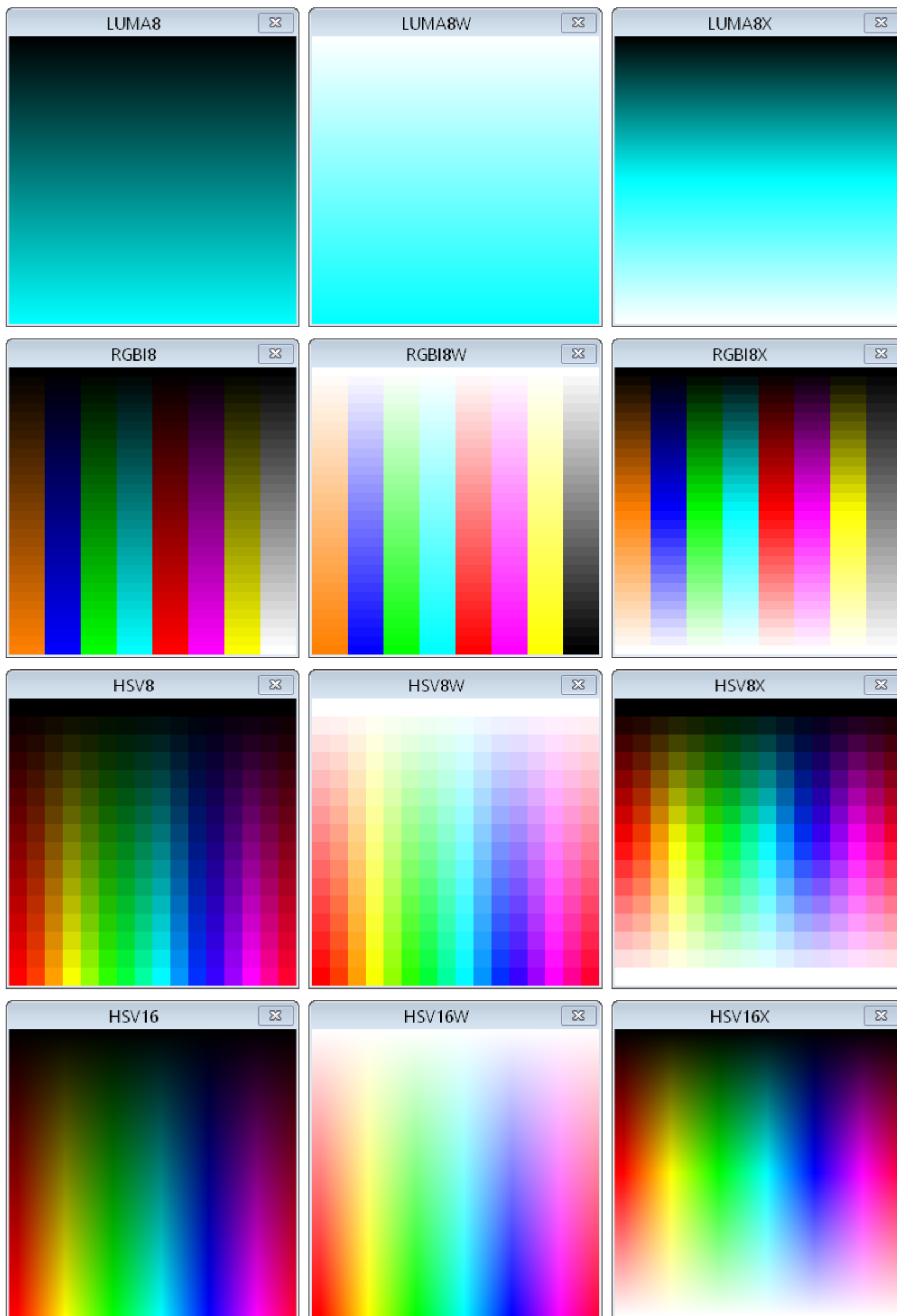
PUB go() | i
  debug(`bitmap a title 'LUT1'   pos 100 100 trace 2 lut1 longs_1bit alt)
  debug(`bitmap b title 'LUT2'   pos 370 100 trace 2 lut2 longs_2bit alt)
  debug(`bitmap c title 'LUT4'   pos 100 395 trace 2 lut4 longs_4bit alt)
  debug(`bitmap d title 'LUT8'   pos 370 395 trace 2 lut8 longs_8bit)
  debug(`bitmap e title 'RGB8'    pos 100 690 trace 2 rgb8)
  debug(`bitmap f title 'RGB16'   pos 370 690 trace 2 rgb16)
  debug(`bitmap g title 'RGB24'   pos 640 690 trace 2 rgb24)
  waitms(1000)

  showbmp("a", @image1, $8A, 2, $800) 'send LUT1 image
  showbmp("b", @image2, $36, 4, $1000) 'send LUT2 image
  showbmp("c", @image3, $8A, 16, $2000) 'send LUT4 image
  showbmp("d", @image4, $36, 256, $4000) 'send LUT8 image

  i := @image5 + $36 'send RGB8/RGB16/RGB24 images from the same 24-bpp file
  repeat $10000
    debug(`e `uhex_(byte[i+0] >> 6 + byte[i+1] >> 5 << 2 + byte[i+2] >> 5 << 5 ))
    debug(`f `uhex_(byte[i+0] >> 3 + byte[i+1] >> 2 << 5 + byte[i+2] >> 3 << 11))
    debug(`g `uhex_(byte[i+0] + byte[i+1] << 8 + byte[i+2] << 16 ))
    i += 3

PRI showbmp(letter, image_address, lut_offset, lut_size, image_long) | i
  image_address += lut_offset
  debug(`#(letter) lutcolors `uhex_long_array_(image_address, lut_size))
  image_address += lut_size << 2 - 4
  repeat image_long
    debug(`#(letter) `uhex_(long[image_address += 4]))

DAT
image1 file "bird_l
```



```

CON _clkfreq = 100_000_000

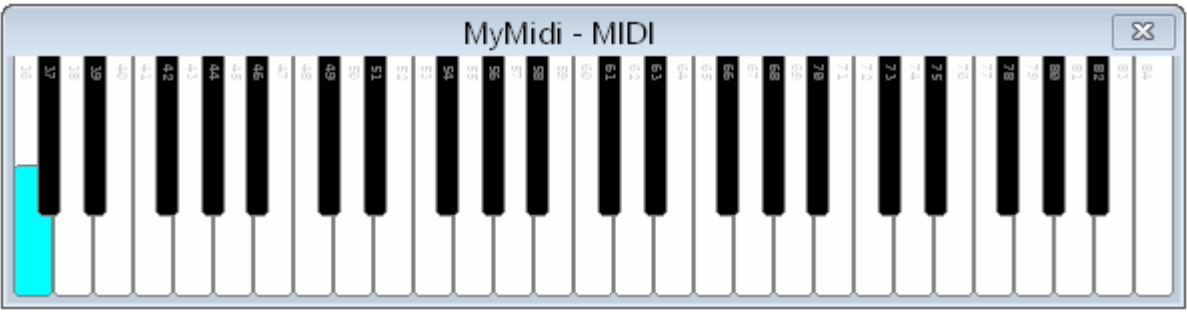
PUB go() | i
  debug(`bitmap a title 'LUMA8' pos 100 100 size 1 256 dotsize 256 1 luma8 cyan)
  debug(`bitmap b title 'LUMA8W' pos 370 100 size 1 256 dotsize 256 1 luma8w cyan)
  debug(`bitmap c title 'LUMA8X' pos 640 100 size 1 256 dotsize 256 1 luma8x cyan)
  debug(`bitmap d title 'RGBI8' pos 100 395 size 8 32 dotsize 32 8 trace 4 rgbi8)
  debug(`bitmap e title 'RGBI8W' pos 370 395 size 8 32 dotsize 32 8 trace 4 rgbi8w)
  debug(`bitmap f title 'RGBI8X' pos 640 395 size 8 32 dotsize 32 8 trace 4 rgbi8x)
  debug(`bitmap g title 'HSV8' pos 100 690 size 16 16 trace 4 dotsize 16 hsv8)
  debug(`bitmap h title 'HSV8W' pos 370 690 size 16 16 trace 4 dotsize 16 hsv8w)
  debug(`bitmap i title 'HSV8X' pos 640 690 size 16 16 trace 4 dotsize 16 hsv8x)
  debug(`bitmap j title 'HSV16' pos 100 985 size 256 256 trace 4 hsv16)
  debug(`bitmap k title 'HSV16W' pos 370 985 size 256 256 trace 4 hsv16w)
  debug(`bitmap l title 'HSV16X' pos 640 985 size 256 256 trace 4 hsv16x)
  waitms(1000)
  repeat i from 0 to 255
    debug(`a b c d e f g h i `uhex_(i))
  repeat i from 0 to 65535
    debug(`j k l `uhex_(i))

```

MIDI 显示

用于查看音符上 / 下状态及音量的 MIDI 键盘

```
控制 CON_clkfreq = 10_000_000 PUB go() | i 调试 (`
midi MyMidi 尺寸 3 范围 36 84) 重复 i 从 36 到 84 重复
调试 (` MyMidi $90 `(i, getrnd() & $7F)) 等待 150 调试
(` MyMidi $80 `(i, 0))
```



MIDI 初始化	描述	默认
标题 ' 字符串 '	将窗口标题设置为 “ 字符串 ”。	无
位置 左上	设置窗口位置。	0, 0
尺寸 键盘尺寸	设置 MIDI 键盘显示尺寸 (1..50)。	4
范围 起始键 结束键	设置起始和结束的 MIDI 键号 (0..127)。	21, 108 (88 个键)
通道 通道号	将 MIDI 通道号设置为观察 (0..15)。	0
白色白键 黑键	为白色和黑色键设置 “ON” 颜色。	青色，品红色
MIDI 输入	描述	默认
字节	如果 \$90 + 通道，则为音符开启模式；否则，为音符关闭模式。 如果进入音符开启模式，则接收一个键（\$00.\$7F），然后是它的音量（\$00.\$7F），更新显示。 如果进入音符关闭模式，则接收一个键(\$00..\$7F)，并更新其音量(\$00..\$7F) 显示。	
清空	清空所有笔记。	
保存 窗口 ' 文件名 '	保存整个窗口或仅显示区域的位图文件（.bmp）。	
关闭	关闭窗口。	

* Color is BLACK / WHITE or ORANGE / BLUE / GREEN / CYAN / RED / MAGENTA / YELLOW / GREY followed by an optional 0..15 for brightness (default is 8).

Here is a PASM program which receives MIDI serial on P16 and sends it to the MIDI display:

```
控制    时钟频率      = 10_000_000
        输入引      = 16
        脚
数据    组织

        汇编时钟

        调试 (midi m 大小 2)

        写入引脚 输入引脚
        写入输出引脚 # ( 时钟频率 /31250) << 16 + 8-1, 输入引脚
        驱动程    输入引脚
        序


```

紧凑数据模式

紧凑数据模式用于高效地传输小于字节的数据类型，通过主机侧对其接收到的字节、字节或长字节进行解包来实现。此外，字节可以在字节和长字节之间发送，并且字节可以在长字节之间发送，以提高效率。

为了建立紧凑数据操作，您必须指定以下任一模式，后跟可选的 “ALT” 和 “SIGNED” 关键词：

紧凑模式 ALT SIGNED

ALT 关键字会导致发送的每个字节内的位、双位或 nibble 在主机侧重新排序。这简化了原始数据在与您使用的调试显示相比时，其位字段顺序不正确的情况。这最有可能适用于以标准格式组成的位图数据。

SIGNED 关键字会导致所有未解包的数据值在主机侧进行符号扩展。

紧凑数据模式	描述	最终值	最终值 如果 SIGNED
长	接收到的每个值都翻译成 32 个单独的 1 位值，从接收值的最低有效位开始。	0..1	-1..0
长	接收到的每个值都翻译成 16 个单独的 2 位值，从接收到的值的最低有效位开始。	0..3	-2..1
长	接收到的每个值都翻译成 8 个单独的 4 位值，从接收值中的最低有效位开始。	0..15	-8..7
长	接收到的每个值都翻译成 4 个单独的 8 位值，从接收值的最低有效位开始。	0..255	-128..127
16 位长整型	接收到的每个值都翻译成 2 个独立的 16 位值，从接收值的最低有效位开始。	0..65,535	-32768..32767
1 位字型	接收到的每个值都翻译成 16 个单独的 1 位值，从接收值的最低有效位开始。	0..1	-1..0
2 位字型	接收到的每个值都翻译成 8 个单独的 2 位值，从接收值中的最低有效位开始。	0..3	-2..1
4 位元数据模式	接收到的每个值都翻译成 4 个单独的 4 位值，从接收值的最低有效位开始。	0..15	-8..7
8 位	接收到的每个值都翻译成 2 个单独的 8 位值，从接收值的最低有效位开始。	0..255	-128..127
字节_1 位	接收到的每个值都翻译成 8 个单独的 1 位值，从接收值的最低有效位开始。	0..1	-1..0
2 位字节	接收到的每个值都翻译成 4 个单独的 2 位值，从接收到的值的最低有效位开始。	0..3	-2..1
4 位字节	接收到的每个值都翻译成 2 个单独的 4 位值，从接收值的最低有效位开始。	0..15	-8..7

智能别针配置的内置符号

智能别针 符号 值	符号名称	详情
A 输入 极性	打包数据模式	
%0000_0000_000_000000000000_00_00000_0	真 (默认)	真 A 输入
%1000_0000_000_000000000000_00_00000_0	反转 A	反转 A 输入
输入选择	打包数据模式	
%0000_0000_000_000000000000_00_00000_0	A 的本地密码 (默认)	选择 A 输入的本地针脚
%0001_0000_000_000000000000_00_00000_0	A 的密码 +1	选择 A 输入中的针脚 +1
%0010_0000_000_000000000000_00_00000_0	A 的密码 +2	选择 A 输入中的针脚 +2
%0011_0000_000_000000000000_00_00000_0	A 的密码 +3	选择 A 输入中的针脚 +3
%0100_0000_000_000000000000_00_00000_0	P_ 输出 _A	选择 A 输入的输出位
%0101_0000_000_000000000000_00_00000_0	P_MINUS3_A	选择 引脚 -3 用于 A 输入
%0110_0000_000_000000000000_00_00000_0	P_MINUS2_A	选择 引脚 -2 用于 A 输入
%0111_0000_000_000000000000_00_00000_0	P_MINUS1_A	选择 引脚 -1 用于 A 输入
B 输入 极性	打包数据模式	
%0000_0000_000_0000000000 0000_00_00000_0	真 (默认)	真 B 输入
%0000_1000_000_000000000000_00_00000_0	反转 B	反转 B 输入
B 输入选择	打包数据模式	
%0000_0000_000_000000000000_00_00000_0	P_LOCAL_B (默认)	为 B 输入选择本地 PIN
%0000_0001_000_000000000000_00_00000_0	P_PLU+1_B	选择 B 输入的引脚 +1
%0000_0010_000_000000000000_00_00000_0	P_PLUS2_B	选择 B 输入的引脚 +2

%0000_0000_000_1000100000000_00_00000_0	P_ADC_FLOAT	ADC FLOAT → IN, output OUT
%0000_0000_000_1000110000000_00_00000_0	P_ADC_1X	ADC 1x → IN, output OUT
%0000_0000_000_1001000000000_00_00000_0	P_ADC_3X	ADC 3.16x → IN, output OUT
%0000_0000_000_1001010000000_00_00000_0	内部时钟模数转换器 (nèibù shízhōng módù zhuǎnhuàn qì)	模数转换器 10x → 输入, 输出 输出
%0000_0000_000_1001100000000_00_00000_0	内部时钟模数转换器 (nèibù shízhōng módù zhuǎnhuàn qì)	模数转换器 31.6x → 输入, 输出 输出
%0000_0000_000_1001110000000_00_00000_0	内部时钟模数转换器 (nèibù shízhōng módù zhuǎnhuàn qì)	模数转换器 100x → 输入, 输出 输出
%xxxx_xxxx_xxx_xxxxxOHHHLLL_xx_xxxx_x	shízhōng módù zhuǎnhuàn qì)	输出极性, 高 / 低驱动
数模转换器输出模式		DIR 启用输出, OUT 启用模数转换器
%0000_0000_000_1010000000000_00_00000_0	P_DAC_990R_3V	数模转换器 990Ω, 3.3V 峰值, 模数转换器 1x → 输入
%0000_0000_000_1010100000000_00_00000_0	P_DAC_600R_2V	数模转换器 600Ω, 2.0V 峰值, 模数转换器 1x → 输入
%0000_0000_000_1011000000000_00_00000_0	P_DAC_124R_3V	数模转换器 123.75Ω, 3.3V 峰值, ADC 1x → 输入
%0000_0000_000_1011100000000_00_00000_0	P_DAC_75R_2V	数模转换器 75Ω, 2.0V 峰值, 模数转换器 1x → 输入
%xxxx_xxxx_xxx_xxxxxDDDDDDDD_xx_xxxx_x		8 位 DAC 值
级比较模式		DIR 启用输出 (1.5kΩ drive)
%0000_0000_000_1100000000000_00_00000_0	P_级 A	电平输入, 输出

%0000_0000_000_00000000000111_00_00000_0	低浮点数	浮点数 低
输出控制	打包数据模式	
%0000_0000_000_000000000000_00_00000_0	P_TT_00 (默认)	TT
%0000_0000_000_000000000000_01_00000_0	P_TT_01	TT =
%0000_0000_000_000000000000_10_00000_0	TT_10	TT = %10
%0000_0000_000_000000000000_11_00000_0	P_TT_11	TT = %11
%0000_0000_000_000000000000_01_00000_0	P_ OE	启用智能引脚模式输出
%0000_0000_000_000000000000_01_00000_0	P_CHANNEL	在非智能别针 DAC 模式下启用 DAC 通道
%0000_0000_000_000000000000_10_00000_0	P_BITDAC	在非智能引脚 DAC 模式下启用 BITDAC
智能别针模式	打包数据模式	
%0000_0000_000_000000000000_00_00000_0	正常模式 (默认)	正常模式 (非智能别针模式)
%0000_0000_000_000000000000_00_00001_0	P_ 仓库	长存储器 (非数模转换器模式)
%0000_0000_000_000000000000_00_00001_0	数模转换器噪声	数模转换器噪声 (DAC 模式)
%0000_0000_000_000000000000_00_00010_0	DAC 抖动随机	16 位随机抖动 (DAC 模式)
%0000_0000_000_000000000000_00_00011_0	P_DAC_ 抖动 PWM	16 位 DAC PWM 抖动 (DAC 模式)
%0000_0000_000_00000000000000_00_00100_0	P_ 脉冲	脉冲 / 周期输出
%0000_0000_000_00000000000000_00_00101_0	P_ 切换	切换输出
%0000_0000_000_000000000000_00_00110_0	P_ 非线性数字控制频率	非线性数字控制频率输出
%0000_0000_000_00000000000000_00_00111_0	P_NCO_DUTY	NCO 职责

流媒体模式的内置符号

符号值	符号名称
即时 → LUT → 引脚 / 数模转换器	
%0000_0000_0000_0000 << 16 %0000_DDDD_EPPP_BBBB << 16	X_IMM_32X1_LUT
%0001_0000_0000_0000 << 16 %0001_DDDD_EPPP_BBBB << 16	X_IMM_16X2_LUT
%0010_0000_0000_0000 << 16 %0010_DDDD_EPPP_BBBB << 16	X_IMM_8X4_LUT
%0011_0000_0000_0000 << 16 %0011_DDDD_EPPP_BBBB << 16	X_IMM_4X8_LUT
立即引脚 / 数模转换器	
%0100_0000_0000_0000 << 16 %0100_DDDD_EPPP_PPPA << 16	X_IMM_32X1_1DAC1
%0101_0000_0000_0000 << 16 %0101_DDDD_EPPP_PP0A << 16	X_IMM_16X2_2DAC1
%0101_0000_0000_0010 << 16 %0101_DDDD_EPPP_PP1A << 16	X_IMM_16X2_1DAC2
%0110_0000_0000_0000 << 16 %0110_DDDD_EPPP_P00A << 16	X_IMM_8X4_4DAC1
%0110_0000_0000_0010 << 16 %0110_DDDD_EPPP_P01A << 16	X_IMM_8X4_2DAC2
%0110_0000_0000_0100 << 16 %0110_DDDD_EPPP_P10A << 16	X_IMM_8X4_1DAC4
%0110_0000_0000_0110 << 16 %0110_DDDD_EPPP_0110 << 16	X_IMM_4X8_4DAC2
%0110_0000_0000_0111 << 16 %0110_DDDD_EPPP_0111 << 16	X_IMM_4X8_2DAC4
%0110_0000_0000_1110 << 16 %0110_DDDD_EPPP_1110 << 16	X_IMM_4X8_1DAC8
%0110_0000_0000_1111 << 16 %0110_DDDD_EPPP_1111 << 16	X_IMM_2X16_4DAC4
%0111_0000_	